Lecture 3. Data Wrangling

dplyr verbs, grouping and summaries, joins, the Uber example

Contents

3.1 A grammar of data wrangling **2**

3.1.1 The single-table verbs 2

3.2 Relational data **4**

3.3 Example: Uber pickups in New York **5**

3.4 Selected exercises **5**

3.1 A grammar of data wrangling

Motivation. dplyr gives a grammar for data: a few verbs, each taking and returning a data frame, chained together.

`dplyr` is fast on in-memory data and separates operations from the data source (the same code runs on a database). It works on data frames and **tibbles** (class `tbl_df`), which print only the first rows and fitting columns, with types. Vignette: `vignette("dplyr", package = "dplyr")`.

We use `ggplot2::presidential`:

```
presidential
```

```
## # A tibble: 12 x 4
##   name       start      end        party
##   <chr>      <date>     <date>     <chr>
## 1 Eisenhower 1953-01-20 1961-01-20 Republican
## 2 Kennedy    1961-01-20 1963-11-22 Democratic
## 3 Johnson    1963-11-22 1969-01-20 Democratic
## 4 Nixon      1969-01-20 1974-08-09 Republican
## 5 Ford       1974-08-09 1977-01-20 Republican
## # i 7 more rows
```

```
class(presidential)
```

```
## [1] "tbl_df"     "tbl"        "data.frame"
```

3.1.1 The single-table verbs

- `filter()` selects rows; `select()` selects columns;
- `mutate()` adds columns; `arrange()` reorders rows;
- `summarize()` collapses a data frame to a single row, usually together with `group_by()`.

filter() keeps the rows where a logical vector is `TRUE`. Comparisons: `>`, `>=`, `<`, `<=`, `!=`, `==`; set membership `x %in% Y`; and they combine with `&`, `|`, `!`.

```
x <- c(2, 1, 3, 0)
x == 0 | x == 1
```

```
## [1] FALSE  TRUE FALSE  TRUE
```

```
x %in% c(0, 1)
```

```
## [1] FALSE  TRUE FALSE  TRUE
```

```
filter(presidential, party == "Republican")
```

```
## # A tibble: 7 x 4
##   name       start      end        party
##   <chr>      <date>     <date>     <chr>
## 1 Eisenhower 1953-01-20 1961-01-20 Republican
## 2 Nixon      1969-01-20 1974-08-09 Republican
## 3 Ford       1974-08-09 1977-01-20 Republican
## 4 Reagan     1981-01-20 1989-01-20 Republican
## 5 Bush       1989-01-20 1993-01-20 Republican
## # i 2 more rows
```

Missing values are contagious: `NA > 5`, `NA + 10`, `NA == NA` are `NA`. Test with `is.na()`; drop with `filter(!is.na(variable))`.

select() picks columns; helpers `starts_with()`, `ends_with()`, `contains()`, `num_range()` select by name pattern:

```
flights %>% select(ends_with("time"))
```

```
## # A tibble: 336,776 x 5
##   dep_time sched_dep_time arr_time sched_arr_time air_time
##      <int>          <int>    <int>          <int>    <dbl>
## 1      517            515      830            819      227
## 2      533            529      850            830      227
## 3      542            540      923            850      160
## 4      544            545     1004           1022      183
## 5      554            600      812            837      116
## # i 336,771 more rows
```

The pipe. Which Democratic presidents served since Watergate? Nesting `select(filter(presidential, start > 1973 & party == "Democratic"), name)` reads inside out. The pipe `%>%` (Ctrl+Shift+M) passes the left side as first argument: `df %>% filter(cond)` = `filter(df, cond)`:

```
presidential %>%
  filter(start > 1973 & party == "Democratic") %>%
  select(name)
```

```
## # A tibble: 3 x 1
##   name
##   <chr>
## 1 Carter
## 2 Clinton
## 3 Obama
```

mutate() creates variables (term length in years via lubridate):

```
my_presidents <- presidential %>%
  mutate(term_length = interval(start, end) / dyears(1))
```

Save as a new object rather than overwrite external data.

arrange() sorts rows; later keys break ties:

```
my_presidents %>% arrange(desc(term_length), party)
```

```
## # A tibble: 12 x 5
##   name       start      end        party      term_length
##   <chr>      <date>     <date>     <chr>            <dbl>
## 1 Clinton    1993-01-20 2001-01-20 Democratic           8
## 2 Obama      2009-01-20 2017-01-20 Democratic           8
## 3 Eisenhower 1953-01-20 1961-01-20 Republican           8
## 4 Reagan     1981-01-20 1989-01-20 Republican           8
## 5 Bush       2001-01-20 2009-01-20 Republican           8
## # i 7 more rows
```

summarize() and group_by(). `summarize()` reduces columns to one value; `n()` counts rows, `sum(party == "Democratic")` counts `TRUE`s.

```
my_presidents %>%
  summarize(N = n(), first_year = min(year(start)), last_year = max(year(end)),
            num_dems = sum(party == "Democratic"),
            years = sum(term_length), avg_term_length = mean(term_length))
```

```
## # A tibble: 1 x 6
##       N first_year last_year num_dems years avg_term_length
##   <int>      <dbl>     <dbl>    <int> <dbl>           <dbl>
## 1    12       1953      2021        5    68            5.67
```

Units [Supplement]

The slides divide by 365.25 and get 0.186; `term_length` is already in years, so the total is 68 years (1953–2021).

Grouping runs the summary per party:

```
my_presidents %>%
  group_by(party) %>%
  summarize(N = n(), avg_term_length = mean(term_length),
            std_term_length = sd(term_length))
```

```
## # A tibble: 2 x 4
##   party          N avg_term_length std_term_length
##   <chr>      <int>           <dbl>           <dbl>
## 1 Democratic     5            5.6             2.34
## 2 Republican     7            5.71            2.32
```

Average terms hardly differ between the parties.

3.2 Relational data

Motivation. Databases keep each fact in one table, linked by keys; analyses join them back.

Relational data: linked tables. Two-table verbs: *mutating joins* (add variables from matching rows), *filtering joins* (keep/drop rows by match), *set operations*. In `nycflights13`, `flights` stores carrier codes and `airlines` maps them to names. Keys: `flights` connects to `airlines` by `carrier`, to `planes` by `tailnum`, to `airports` by `origin` and `dest`, and to `weather` by `origin` and `year`, `month`, `day`, `hour`.

inner_join() keeps rows with a match in both tables:

```
flights_joined <- flights %>%
  inner_join(airlines, by = c("carrier" = "carrier"))
flights_joined %>% select(carrier, name, flight, origin, dest)
```

```
## # A tibble: 336,776 x 5
##   carrier name                  flight origin dest
##   <chr>   <chr>                  <int> <chr>  <chr>
## 1 UA      United Air Lines Inc.   1545 EWR    IAH
## 2 UA      United Air Lines Inc.   1714 LGA    IAH
## 3 AA      American Airlines Inc.  1141 JFK    MIA
## 4 B6      JetBlue Airways          725 JFK    BQN
## 5 DL      Delta Air Lines Inc.     461 LGA    ATL
## # i 336,771 more rows
```

left_join() keeps all rows of the first table, `NA` where unmatched. Pacific-time airports:

```
airports_pt <- airports %>% filter(tz == -8)
flights %>% inner_join(airports_pt, by = c("dest" = "faa")) %>% nrow()
```

```
## [1] 46324
```

```
flights %>%
  left_join(airports_pt, by = c("dest" = "faa")) %>%
  summarize(num_flights = n(),
            num_flights_pt = sum(!is.na(name)),
            num_flights_not_pt = sum(is.na(name)))
```

```
## # A tibble: 1 x 3
##   num_flights num_flights_pt num_flights_not_pt
##         <int>          <int>              <int>
## 1      336776          46324             290452
```

Inner join: 46 324 west-coast flights; left join: all flights, others with `NA` name. Other joins: `right_join()` keeps all rows of the second table, `full_join()` all rows of both, `semi_join()` the rows of the first table that have a match (without adding columns), and `anti_join()` those that do not.

3.3 Example: Uber pickups in New York

Motivation. Combine, parse, derive, group, summarise, plot.

About 4.5 million NYC Uber pickups, April–September 2014: six CSV files with `Date/Time`, `Lat`, `Lon`, `Base` (company code). Code shown without output.

```
files <- sprintf("data/uber-raw-data-%s14.csv", c("apr", "may", "jun", "jul", "aug", "
    sep"))
data_2014 <- map_dfr(files, read_csv) %>%          # stack the six months
  mutate(dt = mdy_hms(`Date/Time`),
         month = month(dt, label = TRUE), day = day(dt),
         dayofweek = wday(dt, label = TRUE), hour = hour(dt))
```

(Slides: `rbind()`, `as.POSIXct()`; `map_dfr()` and lubridate are shorter [Supplement].) Each question is a grouped count plus bar chart:

```
data_2014 %>%
  count(hour, name = "Total") %>%
  ggplot(aes(hour, Total)) +
  geom_col(fill = "steelblue") +
  scale_y_continuous(labels = scales::comma) +
  ggtitle("Trips every hour")
data_2014 %>%
  count(month, dayofweek, name = "Total") %>%
  ggplot(aes(month, Total, fill = dayofweek)) +
  geom_col(position = "dodge")
```

Trips by hour, month, day and weekday follow this pattern. `Lon` against `Lat` (latitude 40.58–40.92, longitude −74.15 to −73.70) draws a map of New York; colour by `Base` for company areas, labelled via `left_join(by = c("Base" = "base_code"))`.

Overplotting millions of points [Supplement]

For 4.5 million points use `size = 0.1, alpha = 0.05` or 2-d bins (`geom_bin2d()`, `geom_hex()`).

3.4 Selected exercises

Motivation. Join sizes, missing-value logic and grouped means, proved once.

Exercise 3.1 (size of a join)

Let `x` and `y` be tables joined by a key. For a key value k let a_k and b_k be the numbers of rows of `x` and of `y` with key k (by default dplyr treats a missing key as one more key value, so `NA` matches `NA`).
(a) Show that `inner_join(x, y)` has $\sum_k a_k b_k$ rows and `left_join(x, y)` has $\sum_k a_k \max(b_k, 1)$ rows.

(b) Deduce that `left_join(x, y)` has exactly as many rows as `x` if and only if every key value of `x` occurs at most once in `y`.
(c) Show that `semi_join(x, y)` and `anti_join(x, y)` split the rows of `x` into two disjoint parts, so their row counts add up to that of `x`.

Solution [Supplement].

(a) An inner join outputs one row per pair (row of `x`, row of `y`) with equal keys: $a_k b_k$ pairs for key k, disjoint across k, total $\sum_k a_k b_k$. A left join adds one `NA`-filled row per unmatched row of `x`: for key k, $a_k b_k$ rows if $b_k \geq 1$ and a_k if $b_k = 0$, i.e. $a_k \max(b_k, 1)$.

(b) By (a) the difference from $\sum_k a_k$ is $\sum_k a_k\{\max(b_k, 1) - 1\}$, a sum of terms ≥ 0; it is 0 iff $b_k \leq 1$ for every k with $a_k > 0$.

(c) A semi join keeps a row of `x` once if $b_k \geq 1$, an anti join once if $b_k = 0$; exactly one holds, so each row appears once in exactly one result (a semi join never duplicates rows, unlike an inner join with $b_k \geq 2$). ■

Exercise 3.2 (logic of missing values; adapted from R4DS, Exercise 5.2.4.4)

`NA` stands for a value that exists but is unknown. Explain why `NA^0` is 1, `NA | TRUE` is `TRUE` and `FALSE & NA` is `FALSE`, while `NA > 5` and `NA * 0` are `NA`. State the general rule.

Solution [Supplement].

Step 1. (Rule.) R returns a known value iff the result is the same for every value the unknown could take.

Step 2. (Known results.) For every real u (including $\pm\infty$ and `NaN` in R's arithmetic) $u^0 = 1$, so `NA^0` is 1. For a logical u, $u \vee \text{TRUE} = \text{TRUE}$ and $\text{FALSE} \wedge u = \text{FALSE}$ whatever u is.

Step 3. (Unknown results.) `NA > 5` is `TRUE` for $u = 6$, `FALSE` for $u = 4$. For `NA * 0`, $u \cdot 0 = 0$ for finite u but $\infty \cdot 0$ is `NaN`, so the result is undetermined: the rule covers all representable values, not only finite ones. ■

Exercise 3.3 (combining grouped means)

Split n rows into groups $g = 1, \ldots, G$ of sizes n_g with group means $\bar{y}_g$ of a variable y.
(a) Show that the overall mean is $\bar{y} = \sum_g (n_g/n)\, \bar{y}_g$, so that the unweighted mean of the group means equals $\bar{y}$ when all n_g are equal or all $\bar{y}_g$ are equal, but not in general.
(b) Two groups A and B are each split by a binary variable s. In A, $s = 1$ for 80 of 100 rows with mean 10 and $s = 0$ for 20 rows with mean 2; in B, $s = 1$ for 20 of 100 rows with mean 11 and $s = 0$ for 80 rows with mean 3. Compare A and B within each level of s and overall.

Solution [Supplement].

(a) $\sum_i y_i = \sum_g \sum_{i \in g} y_i = \sum_g n_g \bar{y}_g$; divide by $n = \sum_g n_g$. Then

$$\bar{y} - \frac{1}{G} \sum_g \bar{y}_g = \sum_g \Big(\frac{n_g}{n} - \frac{1}{G} \Big) \bar{y}_g,$$

which is 0 for equal sizes or for $\bar{y}_g \equiv c$ (then $c \sum_g (n_g/n - 1/G) = 0$), but not in general.

(b) Within levels B is higher ($11 > 10$, $3 > 2$). Overall, by (a), A: $0.8 \cdot 10 + 0.2 \cdot 2 = 8.4$, B: $0.2 \cdot 11 + 0.8 \cdot 3 = 4.6$, so A is higher. The reversal (Simpson's paradox) comes from the different shares of the high-mean level $s = 1$. ■